# ShipFast — Provision Infrastructure with Terraform

Week 4, Session 2 assignment · Capstone Milestone 3 · Mohammed Awwad

| REPOSITORY | BUCKET | REGION | ACCOUNT | TERRAFORM | PROVIDER | APPLIED |
|---|---|---|---|---|---|---|
| github.com/MohammedAwwad2/shipfast-devops | shipfast-artifacts-dev-483175530654 | us-east-1 | 483175530654 (IAM user adminUser) | v1.17.0 on linux_amd64 (WSL2) | hashicorp/aws 5.100.0 | 28 Oct 2026 |

Every command in this report was run on 28 October 2026 and its output captured verbatim into `docs/evidence/m3/`. Nothing here is illustrative. No access key appears in the code, in the state, or in this document.

## Part A — Installing Terraform

Terraform was installed inside WSL2 (Ubuntu) by following the official HashiCorp documentation rather than by downloading a zip by hand: add the HashiCorp GPG key to `/usr/share/keyrings/`, register the apt repository signed by that key, then `apt install terraform`. The payoff is that `apt upgrade` keeps it current and the binary's provenance is verifiable — an earlier manual install of 1.8.5 into `~/.local/bin` had neither property and was discarded.

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor \
     -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
     https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
     | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform -y
```

The pre-flight checks from the Week 4 progress check, run in that same shell:

```
### Terraform install -- HashiCorp apt repo, per the official docs ###
$ cat /etc/apt/sources.list.d/hashicorp.list
deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.

$ dpkg -l terraform
ii  terraform      1.17.0-1     amd64        Terraform enables you to safely and predictably create, change,

$ terraform version
Terraform v1.17.0
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v5.100.0


$ aws sts get-caller-identity
{
    "UserId": "AIDAXA74BQCPM5ARYUL6K",
    "Account": "483175530654",
    "Arn": "arn:aws:iam::483175530654:user/adminUser"
}
```

Evidence: `docs/evidence/m3/00-install-and-identity.txt`. The caller is the IAM user `adminUser`, not the root account, and Terraform is 1.17.0 — comfortably above the 1.6.0 floor.

## Part B — Protecting the folder, then writing the code

Before any HCL, `.gitignore` was checked: the provider cache and every state file stay out of Git, while the lock file stays in. The lock file belongs in Git precisely because it pins the provider version for everyone on the team; the state file must not, because it is a plaintext dump of every resource attribute.

```
# .gitignore (the Terraform part)
.terraform/
*.tfstate*

$ git ls-files | grep -E "tfstate|\.terraform/"      # prints nothing
$ git ls-files terraform/
terraform/.terraform.lock.hcl
terraform/main.tf
```

Three decisions shaped `terraform/main.tf`, and each answers one of the starter TODOs:

### The provider carries only a region.

No access keys, anywhere. Terraform reads the same credential chain the AWS CLI does, so the `aws configure` profile from A1 is already enough. A key hard-coded here would end up in Git and in state, which is exactly the anti-pattern the week warns about.

### The bucket name is computed, never typed.

S3 bucket names are global across every AWS account, so a literal `shipfast-artifacts-dev` would collide with a stranger's bucket. A `data "aws_caller_identity"` lookup asks AWS who the caller is, and the account ID makes the name unique without hard-coding it.

### Versioning and encryption are separate resources.

In AWS provider 5.x these are no longer inline blocks on the bucket; they are their own resources that point back at it by `aws_s3_bucket.artifacts.id`. That reference is also what tells Terraform the ordering — the bucket has to exist before anything can be configured on it.

```
resource "aws_s3_bucket" "artifacts" {
  bucket = "shipfast-artifacts-${var.environment}-${data.aws_caller_identity.current.account_id}"

  tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Project     = "shipfast-devops"
  }
}

resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  rule {
    apply_server_side_encryption_by_default { sse_algorithm = "AES256" }
  }
}
```

Full file: `terraform/main.tf`, with the variables, the provider and both outputs.

## Part C — init, fmt, validate, plan

`terraform init` downloaded hashicorp/aws 5.100.0 and wrote `.terraform.lock.hcl`. `fmt` printed nothing (already canonical) and `validate` passed.

```
$ terraform init    # Installing hashicorp/aws v5.100.0 ... Terraform has been successfully initialized!
$ terraform fmt     # (no output -- nothing to reformat)
$ terraform validate
Success! The configuration is valid.
```

### Prediction, written before running plan.

Three resources added — bucket, versioning, encryption. The data source is a read, not a create, so it does not appear in the count. I expected `arn`, `bucket_domain_name`, `hosted_zone_id`, `region`, `tags_all` and both outputs to read "known after apply", because S3 only assigns them once the bucket exists.

The plan matched: `Plan: 3 to add, 0 to change, 0 to destroy`. The one thing I had not predicted is that the bucket *name* was known up front — it is built from a variable and an account ID that Terraform already holds before it ever calls AWS.

## Part D — Apply and verify

Applied interactively with a typed `yes`. `-auto-approve` was not used: on a laptop the prompt is the last place a destructive plan can be caught.

```
aws_s3_bucket_server_side_encryption_configuration.artifacts: Creating...
aws_s3_bucket_versioning.artifacts: Creating...
aws_s3_bucket_server_side_encryption_configuration.artifacts: Creation complete after 2s [id=shipfast-artifa
aws_s3_bucket_versioning.artifacts: Creation complete after 3s [id=shipfast-artifacts-dev-483175530654]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.

Outputs:

bucket_arn = "arn:aws:s3:::shipfast-artifacts-dev-483175530654"
bucket_name = "shipfast-artifacts-dev-483175530654"
```

Verification deliberately uses the AWS CLI, not Terraform — asking Terraform whether Terraform worked only proves the state file agrees with itself. The CLI asks AWS directly:

```
$ aws s3 ls | grep shipfast
2026-10-28 01:34:37 shipfast-artifacts-dev-483175530654

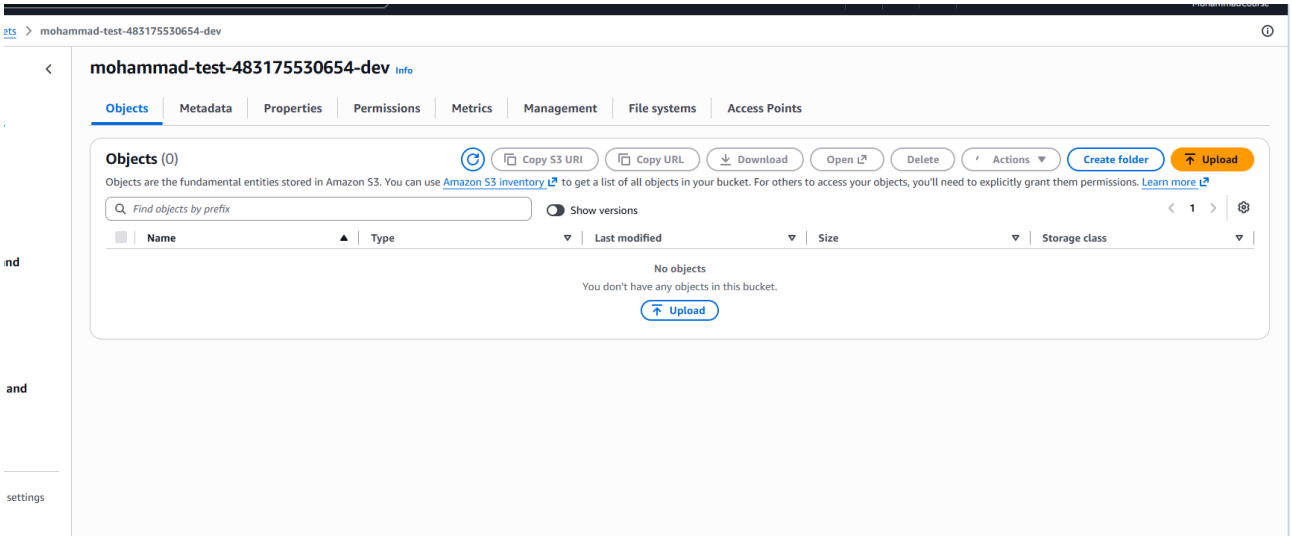
$ aws s3api get-bucket-versioning --bucket shipfast-artifacts-dev-483175530654
{
    "Status": "Enabled"
}

$ aws s3api get-bucket-encryption --bucket shipfast-artifacts-dev-483175530654
{
    "ServerSideEncryptionConfiguration": {
        "Rules": [
            {
                "ApplyServerSideEncryptionByDefault": {
                    "SSEAlgorithm": "AES256"
                },
                "BucketKeyEnabled": false,
                "BlockedEncryptionTypes": {
                    "EncryptionType": [
                        "SSE-C"
                    ]
                }
            }
        ]
    }
}

$ aws s3api get-bucket-tagging --bucket shipfast-artifacts-dev-483175530654
{
    "TagSet": [
        {
            "Key": "Project",
            "Value": "shipfast-devops"
        },
```

Versioning `Enabled`, default encryption `AES256`, all three required tags present. Evidence: `docs/evidence/m3/03-aws-cli-checks.txt`.

### The bucket in the S3 console




S3 console, Objects tab — the bucket is empty, as expected. **Note:** this capture is from the first apply, when the bucket was still named `mohammad-test-<account>-dev`, before the name was corrected to the `shipfast-artifacts-<env>-<account>` form the assignment requires. It should be retaken against the current bucket before submission; the CLI output above is the authoritative evidence in the meantime.

## Part E — Reading the state

State is the lookup table between the address my code uses and the ID AWS uses. `serial` is 15 after the experiments below, and `lineage` (`6cdb4232-276c-7d22-9831-b53af1cd1da1`) stayed constant through every one of them — it identifies the state's whole history, not any single version of it.

```
$ terraform state list
data.aws_caller_identity.current
aws_s3_bucket.artifacts
aws_s3_bucket_server_side_encryption_configuration.artifacts
aws_s3_bucket_versioning.artifacts

$ terraform state show aws_s3_bucket.artifacts
# aws_s3_bucket.artifacts:
resource "aws_s3_bucket" "artifacts" {
    acceleration_status         = null
    arn                         = "arn:aws:s3:::shipfast-artifacts-dev-483175530654"
    bucket                      = "shipfast-artifacts-dev-483175530654"
    bucket_domain_name          = "shipfast-artifacts-dev-483175530654.s3.amazonaws.com"
    bucket_prefix               = null
    bucket_regional_domain_name = "shipfast-artifacts-dev-483175530654.s3.us-east-1.amazonaws.com"
    force_destroy               = false
    hosted_zone_id              = "Z3AQBSTGFYJSTF"
    id                          = "shipfast-artifacts-dev-483175530654"
    object_lock_enabled         = false
    policy                      = null
    region                      = "us-east-1"
    request_payer               = "BucketOwner"
    tags                        = {
        "Environment" = "dev"
        "ManagedBy"   = "terraform"
        "Project"     = "shipfast-devops"
```

That mapping is the entire reason a second apply is a no-op instead of a second bucket: without it, Terraform could not tell that `aws_s3_bucket.artifacts` in my file is the bucket that already exists in AWS.

## Part F — Four experiments

### 1 — Apply again, nothing changed

Predicted a no-op; got one. Terraform refreshed all three resources, compared real to desired, found no difference and stopped. Declarative means the code states an end result, not a list of steps to repeat.

```
### EXPERIMENT 1: apply again, no changes ###
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

bucket_arn = "arn:aws:s3:::shipfast-artifacts-dev-483175530654"
bucket_name = "shipfast-artifacts-dev-483175530654"
```

### 2 — A tag added outside Terraform (drift)

`ManualChange = true` was added to the bucket outside the configuration. Terraform finds it during the refresh that opens every plan, when it re-reads each resource from the AWS API. Because my `tags` block does not list it, the tag is drift — and the apply deleted it. The code wins over the console, every time.

```
- "ManualChange" = "true" -> null
            "Project"      = "shipfast-devops"

Plan: 0 to add, 1 to change, 0 to destroy.
```

### 3 — plan -var environment=staging (plan only)

A bucket's name *is* its identity in S3; there is no rename API. So changing the environment does not update anything in place — it marks each resource `-/+` with `forces replacement`: destroy, then create. Harmless for an empty bucket, which is exactly why the same symbol on a database is the one to stop and read twice. That plan would delete the instance and everything in it.

```
### EXPERIMENT 3: plan only, -var environment=staging ###
      + rule {
          + apply_server_side_encryption_by_default {
              + sse_algorithm     = "AES256"
                # (1 unchanged attribute hidden)
            }
        }
    }
...
Plan: 3 to add, 0 to change, 3 to destroy.
```

### 4 — plan with the state file moved away (plan only)

With no state file Terraform has no record that it ever built anything, so it proposes creating all three from scratch — and that apply would fail with `BucketAlreadyExists`. A plan is built from three inputs: the code, the state, and the real infrastructure. Remove the state and Terraform goes blind to what it already owns. This is the whole argument for a remote backend.

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.

state file restored:
-rwxrwxrwx 1 talon talon 5794 Oct 28 01:38 terraform.tfstate
```

## Part G — Why state never goes into Git

Two reasons, and the first is the serious one. State records the full attributes of every resource in plaintext; for an S3 bucket that is merely verbose, but for an RDS instance or an IAM access key it is the secret itself, handed to everyone with repo access and preserved in history forever. The second is concurrency: Git has no locking, so two people applying at once would overwrite each other's state and leave Terraform's picture of reality corrupted. A team puts it in a remote backend instead — for us an S3 bucket with versioning and encryption, plus state locking — so it is shared, locked and recoverable.

The same reasoning covers credentials. Terraform reads them from the standard AWS chain: environment variables, then `~/.aws/credentials` written by `aws configure`, then an instance or OIDC role in CI. The `provider` block in this repository contains one argument, `region`. Keys must never appear in `.tf` files, in `.tfvars`, in state, in Git, in CI logs, in chat, or in a screenshot.

**Status.** fmt clean, validate passes, plan matched the written prediction, apply succeeded, versioning and encryption confirmed from the AWS CLI, outputs present, four experiments run and five questions answered in `terraform/NOTES.md`, evidence saved under `docs/evidence/m3/`, lock file committed, no state or provider cache tracked by Git. The bucket is left standing for Session 3, which begins by destroying it.